

Question 1

(a) Compare and Contrast (two approaches in the scenario, select one). You should discuss the pros and cons and choose which is better based on the discussion

1. Systems Development Life Cycle (SDLC)

Overview

- A **phased, structured methodology**, also known as the waterfall approach.
- Development proceeds through **seven phases**:
 1. Identifying problems, opportunities, and objectives
 2. Determining human information requirements
 3. Analyzing system needs
 4. Designing the recommended system
 5. Developing and documenting software
 6. Testing and maintaining the system
 7. Implementing and evaluating the system
- Highly linear; each phase depends on the completion of the previous phase.

Pros

- Highly **organized** and structured; clear deliverables.
- **Comprehensive documentation**, which supports long-term maintenance.
- Formal **testing and evaluation** reduce system failures.
- Suitable for **large-scale, mission-critical systems** with stable requirements.

Cons

- **Inflexible**: hard to incorporate changes once a phase is completed.
- **Slow development**; end-users may not see usable software until late.

- Risk of **misalignment** with user needs if initial requirements are inaccurate.

Best Use

- Projects with **stable, well-defined requirements**.
 - Systems where **safety, compliance, or high reliability** are essential.
-

2. Agile Methodologies

Overview

- **Iterative and incremental approach** to development.
- Focuses on **communication, simplicity, feedback, and courage**.
- Core practices include:
 - Short development cycles (sprints, typically 2–4 weeks)
 - Continuous user involvement
 - Incremental delivery and testing
 - Rapid adaptation to change
- Agile stages: Exploration → Planning → Iterations → Productionizing → Maintenance.

Pros

- **Highly flexible**, allowing requirement changes during development.
- Users **interact with working versions early**, improving satisfaction.
- Enables **quick delivery** of high-priority features.
- Encourages **collaboration and iterative learning**.

Cons

- Less emphasis on **formal documentation**, which may affect long-term maintainability.
- Requires **continuous customer involvement**, which can be resource-intensive.
- Risk of **scope creep** if priorities are not carefully managed.
- May be less suitable for **large, highly regulated systems**.

Best Use

- Projects with **dynamic, evolving requirements**.
- Systems that benefit from **incremental, user-driven development**.
- Teams comfortable with **iterative, adaptive methods**.

Recommended Approach: Agile with Selected Features Prototyping

- Users can **test features early**, providing feedback to shape the system.
- Rapid delivery of **essential features first**, keeping budget and time constraints in mind.
- Easier adaptation to **changing business needs** and user priorities.
- Prototype can serve as **incremental part of final system**, reducing risk.

SDLC may be too rigid and slow for a small, dynamic project.

(b) Prototyping techniques. Based on 1(a), there are multiple techniques, evaluate each and then pick the suitable one.

1. Patched-Up Prototype

Definition

A patched-up prototype is a **working model** that has all the basic features of the system but is built in a quick, inefficient, or “patched-together” way. In engineering, this is called *breadboarding*: wiring together temporary parts to make a circuit work. In information systems, it means building applications rapidly so that they function but may perform slowly or lack proper optimization.

Strengths

- Allows users to **interact with a functioning system**, not just a design.
- Helps users become familiar with the **interface and available outputs**.
- Provides **realistic testing of workflows and logic**.
- Demonstrates that the concept is **feasible** before spending resources on efficiency.

Weaknesses

- The system is often **inefficient and fragile**.
- If users mistake it for the final system, they may become frustrated when performance is poor.
- Creates **technical debt** if developers build on the patched code instead of refactoring.
- May ignore **security and scalability**.

Use Cases

- When the goal is to **validate workflows and logic** before worrying about efficiency.
- Small, straightforward systems where speed of delivery matters more than optimization.
- When management wants to “see it working” quickly.

Examples

- A bank’s **loan processing demo**, where loan approval logic works but the database runs very slowly.
- A **university library system** where check-in/check-out is functional, but search queries take minutes.
- A **start-up’s payroll system** that produces correct salaries but uses clumsy spreadsheets for storage.
- **Pyramid, Inc.’s warehouse inventory system**, where order entry and stock updates work but reports take long to generate.

2. Nonoperational Prototype

Definition

A nonoperational prototype is a **nonworking scale model** that focuses on input/output and interface design, but does not process real data. It is similar to building a **full-size model of a car** for wind tunnel testing: it looks real, but it does not drive. In IS, this often means prototyping only the forms, screens, and reports while ignoring backend processing.

Strengths

- Quick and **low-cost** to create.
- Excellent for **UI/UX testing** — users can judge how easy it is to navigate, read reports, or enter data.
- Helps bridge the gap between **abstract requirements** and what users actually expect.
- Reduces the risk of **miscommunication** between analysts and users.

Weaknesses

- Provides **no real functionality** — users cannot test performance or processing.
- May cause **disappointment** when users realize the prototype does not “work.”
- Cannot test **integration with databases or external systems**.

Use Cases

- When backend coding is too costly or premature.
- When design and usability are the **main priority**.
- For stakeholder approval of a form, layout, or screen flow before investing in development.

Examples

- **Clone Bank’s monthly account summary form**: only the layout matters, not backend calculations.

- **Airline booking system mockup**: users test seat selection screens, but no real booking happens.
 - **Mobile app wireframe** in Figma or Adobe XD, where buttons can be clicked but no real action occurs.
 - A **government portal prototype** where citizens try filling in tax forms, but no backend validation runs.
-

3. First-of-a-Series Prototype

Definition

A first-of-a-series prototype, often called a **pilot**, is a **full-scale, operational version** of the system implemented in one location or for one group of users. It is equivalent to building the first airplane in a new series and testing whether it flies. If successful, it becomes the template for full rollout.

Strengths

- Provides **realistic, production-level testing** in actual working conditions.
- Identifies **integration issues** early, before widespread deployment.
- Builds **user confidence**, since it is a real, working system.
- Allows training and feedback under real-world conditions.

Weaknesses

- Expensive and time-consuming.
- High risk if the pilot fails — could damage user trust.
- Less flexible for major changes, because it is already close to the final product.

Use Cases

- Large-scale projects where the system will be **replicated across multiple sites**.
- When management wants a **real-world trial** before full investment.

- When mission-critical systems require **rigorous testing** in realistic environments.

Examples

- A **retail grocery chain** piloting blockchain shipment tracking in one store.
 - A **university ERP system** tested in one faculty before full university rollout.
 - A **hospital management system** tested in one wing before full deployment.
 - A **bank's ATM software** tested in one city branch before deployment across the network.
-

4. Selected-Features Prototype

Definition

A selected-features prototype is an **operational model** that includes some, but not all, features of the final system. It is built **in modules**, so successful features can be incorporated into the final system. This is like a shopping mall opening with only a few shops ready, while others are still under construction.

Strengths

- Provides users with **usable modules early**.
- Supports **iterative development** and feedback.
- Reduces rework by integrating tested modules into the final system.
- Matches **Agile principles** of delivering working software in increments.

Weaknesses

- Requires **careful prioritization** of features to implement first.
- Users may become frustrated with **incomplete functionality**.
- Risk of **scope creep** if users keep requesting more modules.

Use Cases

- Large modular systems where requirements evolve over time.
- Agile development projects where **incremental delivery** is expected.
- When management wants early value delivered without waiting for full completion.

Examples

- An **HR system** launched with employee records, later adding payroll and performance tracking.
- An **e-learning platform** with video lessons first, then adding quizzes, discussion forums, and grading.
- A **mobile banking app** starting with balance inquiry, then adding funds transfer and bill payment.
- **Pyramid, Inc.'s warehouse system**, beginning with book stock entry, then adding supplier tracking and shipping.

Question 2

(a) Analyze based on scenario, identify 3 problems

1. Outdated or inaccurate documents/records

- Theory : Documents may not reflect current practices; relying solely on recorded procedures can misrepresent workflows.
- Scenario context: Order forms, manuals, or performance reports may be old or incomplete. Analysts might assume procedures are followed exactly as documented, missing recent ad-hoc changes or updated policies.

2. Incomplete or biased data capture

- Theory : Data capture can be inconsistent — fields skipped, forms partially filled, or unofficial “bootleg” forms used.
- Scenario context: Some staff may ignore certain fields, use old forms, or manually adjust entries. This leads to partial data analysis and can hide recurring errors in the order processing system.

3. Hidden workarounds and process mismatches

- Theory : People often create shortcuts, alternative workflows, or unofficial forms when official procedures are inefficient. These practices distort the apparent process flow.

- Scenario context: Employees may bypass the documented system, using personal templates or informal approvals. This hides bottlenecks, slows order processing, and increases the risk of errors going unnoticed.

(b) What additional info will identify more problems? (Theory+ why needed, with context from scenario)

Unobtrusive Methods for Data Collection

Theory:

- Unobtrusive methods gather information without directly interacting with participants.
- Unlike interviews or surveys, they do not rely on self-reporting, which can be biased or inaccurate.
- Examples include:
 - Observing documents, records, or logs
 - Monitoring system usage data
 - Examining physical traces (workspace arrangement, equipment usage, posted notices)
 - Reviewing archival data, emails, or historical reports

Why needed / Scenario context:

- People may behave differently if they know they are being observed (Hawthorne effect), so unobtrusive methods capture natural behavior.
- Helps identify hidden problems that don't appear in formal procedures or interviews:
 - Unofficial or "bootleg" forms in use
 - Workarounds or ad-hoc procedures
 - Bottlenecks caused by shared equipment or inefficient layout
- Example: Examining printer logs or workflow patterns reveals that forms are frequently reprinted due to errors, something employees may not admit in interviews.

Advantages:

- Less biased — reflects actual practices rather than intended practices.
- Can uncover issues participants might forget or deliberately hide.
- Often faster and less disruptive than repeated interviews or observations.

1. Quantitative Document Sampling & Analysis

Theory:

- Apply statistical sampling to examine a subset of documents instead of reviewing all records.
- Methods include:
 - **Simple random sampling**: every document has an equal chance of selection.
 - **Systematic sampling**: select every kth document (e.g., every 10th invoice).
 - **Stratified sampling**: divide documents into meaningful groups (e.g., department, month) and sample from each.
- Calculate the **sample size** based on the desired confidence level and acceptable margin of error.

Why needed / Scenario context:

- In a large organization, manually checking all transaction forms is impractical.
- Sampling helps quantify the frequency of errors like missing fields, incorrect codes, or mismatched quantities.
- Converts subjective observations (“I think 10% of forms have errors”) into measurable, statistically valid insights.
- Example: Sampling 500 forms from the past 6 months to determine the actual error rate in order processing.

2. Analyze Data Capture Forms (Official vs. Bootleg)

Theory:

- Collect **both blank and completed forms**, including unofficial “bootleg” versions used by employees.

- Compare official and bootleg forms to identify:
 - Skipped or redundant fields
 - Usability issues (confusing layouts, unclear instructions)
 - Patterns in unofficial form creation (e.g., missing features, repeated workarounds)

Why needed / Scenario context:

- Reveals which fields are consistently skipped, showing design flaws.
 - Uncovers informal practices that staff use to cope with inefficient forms.
 - Helps prioritize **redesign efforts** or additional **training**.
 - Example: If multiple employees create their own tracking sheets, it signals the official form is inadequate for daily tasks.
-

3. Structured Observation of the Environment (STROBE)

Theory:

- Observe decision-makers and staff in their workspace using a systematic checklist:
 - **Office layout:** location, accessibility, and hierarchy clues
 - **Desk placement:** interactions encouraged or blocked
 - **Props/equipment:** PCs, printers, phones, personal items
 - **Information sources:** newspapers, manuals, digital displays
 - **Lighting, color, and environment:** task-focused vs. casual atmosphere
- Record observations to compare with documented workflows.

Why needed / Scenario context:

- Confirms whether documented procedures align with real-world behavior.

- Detects bottlenecks caused by physical environment (e.g., shared terminals slowing form entry).
 - Example: Observing that forms pile up near a single printer indicates workflow inefficiencies not evident in the procedure manual.
-

4. Observe Decision Makers & Playscript Method

Theory:

- Record observed activities as a **playscript**, with:
 - **Actor column**: person being observed
 - **Action column**: verbs describing actions (asks, inputs, prints, sends, reads)
- Captures **dynamic workflow**, interruptions, and information flow.

Why needed / Scenario context:

- Reveals ad-hoc steps, handoffs, and rework that static documents cannot show.
 - Helps identify tasks prone to errors or delays.
 - Example: A manager repeatedly interrupts a clerk to clarify forms — identifying this allows process redesign or delegation.
-

5. Text Analytics on Unstructured Data

Theory:

- Analyze free-text data such as:
 - Emails, chat logs, complaints, helpdesk tickets, social media posts
- Use tools to identify **keywords, co-occurrences, and recurring themes**.
- Visualize insights with concept maps, word clouds, or ranked lists.

Why needed / Scenario context:

- Captures issues employees or customers report that aren't in structured documents.
- Detects frustration points, usability issues, and recurring complaints.
- Example: Analysis shows repeated mentions of "form rejected incorrectly" — indicates a system validation issue or unclear instructions.

6. Sampling for Interviews

Theory:

- Conduct interviews using **purposive or stratified sampling**:
 - Include representatives from different roles, departments, or seniority levels.
 - Ensures diverse perspectives, avoiding bias from interviewing only senior managers.
- Focus on understanding **motivation, workarounds, and process issues**.

Why needed / Scenario context:

- Validates observations and document analyses.
- Identifies cultural, policy, or training-related barriers.
- Example: Interviews reveal that clerks skip certain form fields because software doesn't allow saving incomplete entries — a reason not visible in forms alone.